



G L O B A L R A I N

Practices for Secure Software Report

Table of Contents

DOCUMENT REVISION HISTORY	3
CLIENT.....	3
INSTRUCTIONS.....	3
DEVELOPER	4
1. ALGORITHM CIPHER	4
2. CERTIFICATE GENERATION	4
3. DEPLOY CIPHER.....	6
4. SECURE COMMUNICATIONS	7
5. SECONDARY TESTING.....	7
6. FUNCTIONAL TESTING	8
7. SUMMARY	9
8. INDUSTRY STANDARD BEST PRACTICES.....	9

Document Revision History

Version	Date	Author	Comments
1.0	12/19/25	Oliver Rodriguez	

Client



Instructions

Submit this completed practices for secure software report. Replace the bracketed text with the relevant information. You must document your process for writing secure communications and refactoring code that complies with software security testing protocols.

- Respond to the steps outlined below and include your findings.
- Respond using your own words. You may also choose to include images or supporting materials. If you include them, make certain to insert them in all the relevant locations in the document.
- Refer to the Project Two Guidelines and Rubric for more detailed instructions about each section of the template.

Developer

Oliver Rodriguez

1. Algorithm Cipher

To address the security vulnerabilities and protect sensitive financial data for Artemis Financial, AES-256 and SHA-256 are recommended. AES-256 (Advanced Encryption Standard) is a symmetric encryption algorithm used to protect data confidentiality, while SHA-256 (Secure Hash Algorithm) is a cryptographic hash function used to ensure data integrity through checksum verification. These algorithms are widely accepted as industry standards and are well suited for securing financial web applications.

SHA-256 generates a 256-bit hash value, which makes it highly resistant to collision and preimage attacks. In this project, SHA-256 is used to generate a checksum to verify that data has not been altered during transmission. AES-256 uses a 256-bit encryption key, providing strong protection against brute-force attacks and making it appropriate for encrypting sensitive information such as financial records and client data.

AES-256 is a symmetric encryption algorithm, meaning the same key is used for both encryption and decryption. Secure random number generation is critical when creating encryption keys to prevent predictability and reduce the risk of compromise. SHA-256 does not use encryption keys, as it is a one-way hashing function designed solely for verification rather than encryption. In secure web communications, asymmetric encryption is used during the initial TLS handshake, after which symmetric encryption like AES-256 is used for efficient data transfer.

Older encryption algorithms such as DES, MD5, and SHA-1 are no longer considered secure due to known vulnerabilities and advances in computing power. AES was selected by NIST in 2001 as the replacement for DES and remains the standard for symmetric encryption today. SHA-256 replaced weaker hashing algorithms and continues to be recommended for secure applications. Currently, AES-256 and SHA-256 are trusted and widely used across financial, government, and enterprise systems to protect sensitive data and ensure secure communications.

2. Certificate Generation

Insert a screenshot below of the CER file.

Install the latest PowerShell for new features and improvements!
<https://aka.ms/PSWindows>

```
PS C:\Users\Olive\Downloads\CS 305 Project Two Code Base\ssl-server_student> keytool -genkeypair -keyalg RSA -alias selfsigned -keypass tamashiro -keystore keystore.jks -storepass tamashiro -validity 360 -keysize 2048
```

What is your first and last name?

[Unknown]: Oliver Rodriguez

What is the name of your organizational unit?

[Unknown]: IT

What is the name of your organization?

[Unknown]: Artemis Financial

What is the name of your City or Locality?

[Unknown]: Chicago

What is the name of your State or Province?

[Unknown]: IL

What is the two-letter country code for this unit?

[Unknown]: US

Is CN=Oliver Rodriguez, OU=IT, O=Artemis Financial, L=Chicago, ST=IL, C=US correct?

[no]: yes

```
PS C:\Users\Olive\Downloads\CS 305 Project Two Code Base\ssl-server_student>
```

```

PS C:\Users\Olive\Downloads\CS 305 Project Two Code Base\ssl-server_student> keytool -export -alias selfsigned -storepass tamashiro -file server.cer -keystore keystore.jks
Certificate stored in file <server.cer>
PS C:\Users\Olive\Downloads\CS 305 Project Two Code Base\ssl-server_student> keytool -printcert -file server.cer
Owner: CN=Oliver Rodriguez, OU=IT, O=Artemis Financial, L=Chicago, ST=IL, C=US
Issuer: CN=Oliver Rodriguez, OU=IT, O=Artemis Financial, L=Chicago, ST=IL, C=US
Serial number: 3ca6fe3e
Valid from: Sun Dec 21 09:53:04 CST 2025 until: Wed Dec 16 09:53:04 CST 2026
Certificate fingerprints:
    SHA1: F0:16:3E:75:20:CF:24:B3:A7:2D:8F:C4:33:D2:B0:C7:32:47:7F:D4
    SHA256: 52:F7:83:66:1F:02:D0:04:75:3E:12:F4:6B:29:6E:5E:BD:4E:E5:D6:36:BD:F3:06:C1:56:B9:E9:99:BE:A7:21
Signature algorithm name: SHA256withRSA
Subject Public Key Algorithm: 2048-bit RSA key
Version: 3

Extensions:

#1: ObjectId: 2.5.29.14 Criticality=false
SubjectKeyIdentifier [
KeyIdentifier [
0000: F5 88 39 EC 50 9C 29 5D    41 2D 79 33 1E D5 8A A0    ..9.P.)
]A-y3....
0010: CC 77 AC B3                                .W..
]
]

PS C:\Users\Olive\Downloads\CS 305 Project Two Code Base\ssl-server_student>

```

3. Deploy Cipher

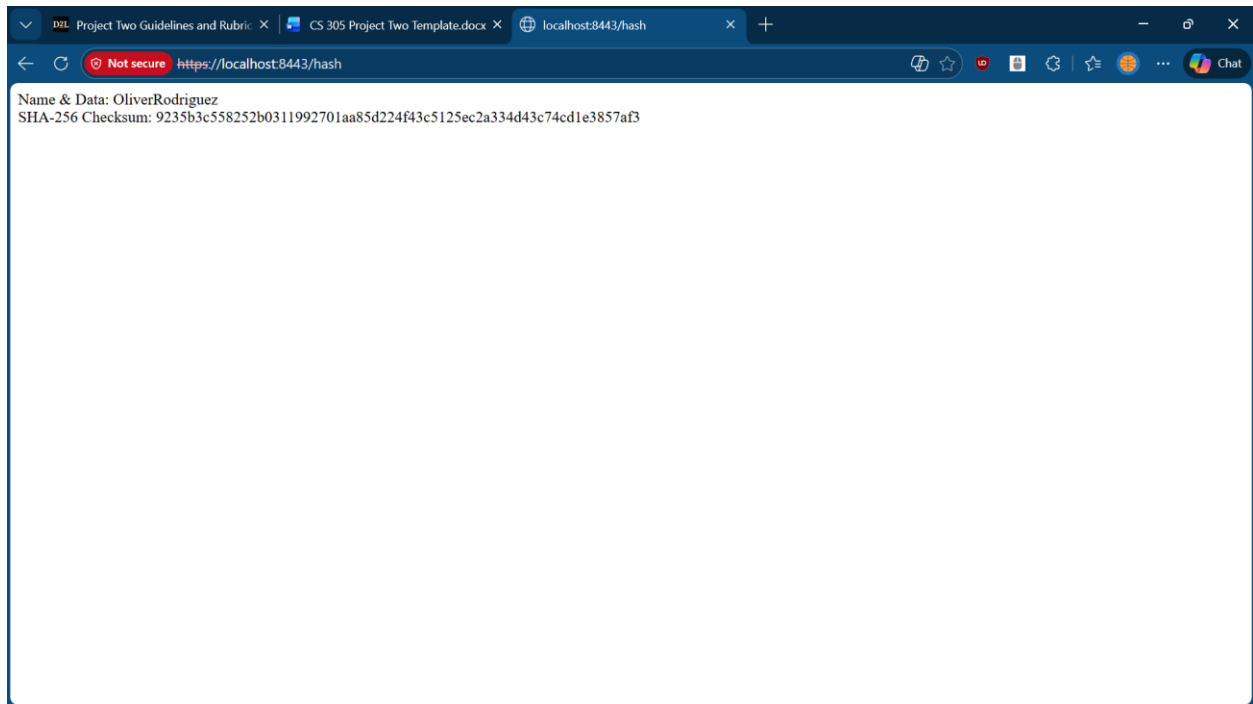
Insert a screenshot below of the checksum verification.

Name & Data: OliverRodriguez

SHA-256 Checksum: 9235b3c558252b0311992701aa85d224f43c5125ec2a334d43c74cd1e3857af3

4. Secure Communications

Insert a screenshot below of the web browser that shows a secure webpage.



5. Secondary Testing

Insert screenshots below of the refactored code executed without errors and the dependency-check report.

```

1 package com.snhu.ssiserver;
2
3 import org.springframework.web.bind.annotation.GetMapping;
4 import org.springframework.web.bind.annotation.RestController;
5
6 @RestController
7 public class HashController {
8
9     @GetMapping("/hash")
10    public String getHash() throws Exception {
11        // Your unique data string (include your name)
12        String data = "OliverRodriguez";
13
14        // Compute SHA-256 checksum
15        String checksum = ChecksumUtil.computeSHA256(data);
16
17        return "Name & Data: " + data + "<br>SHA-256 Checksum: " + checksum;
18    }
19 }

```

Project: ssl-server

com.snhu.ssiserver:0.0.1-SNAPSHOT

Scan Information ([show all](#)):

- dependency-check version: 12.1.9
- Report Generated On: Sun, 21 Dec 2025 11:16:38 -0600
- Dependencies Scanned: 49 (28 unique)
- Vulnerable Dependencies: 13
- Vulnerabilities Found: 147
- Vulnerabilities Suppressed: 0
- ...

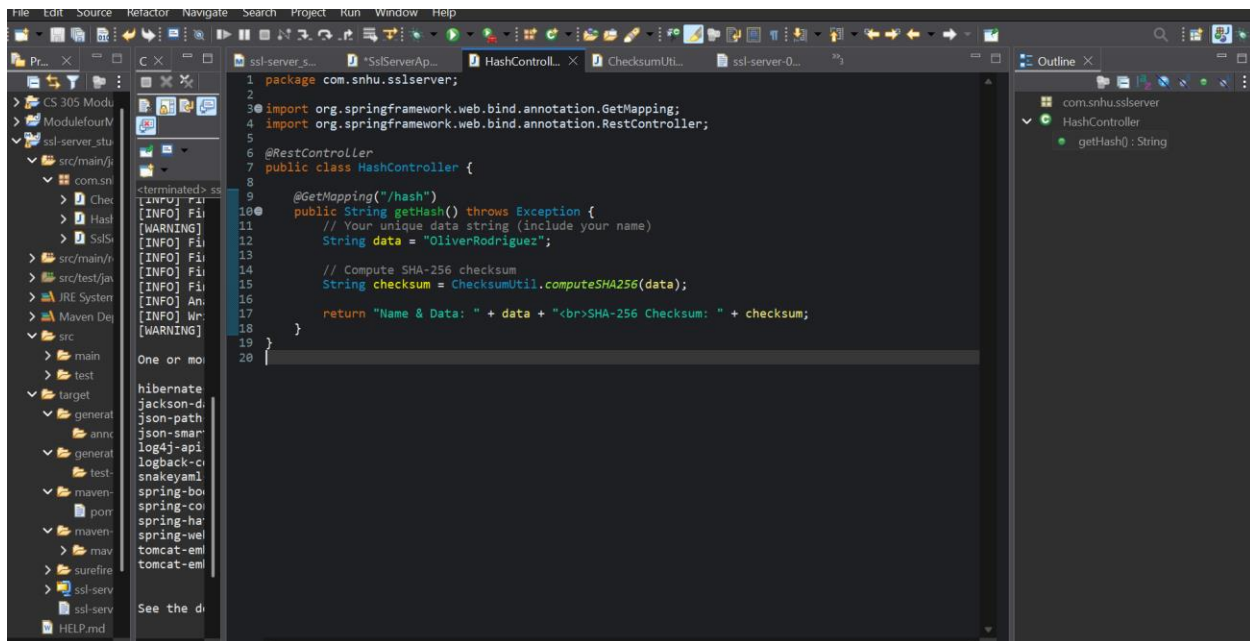
Summary

Summary of Vulnerable Dependencies ([click to show all](#))

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count
hibernate-validator-6.0.18.Final.jar	cpe:2.3:a:hibernate:hibernate-validator:6.0.18:*:*:*:* cpe:2.3:a:redhat:hibernate_validator:6.0.18:*:*:*	pkg:maven/org.hibernate.validator/hibernate-validator@6.0.18.Final	MEDIUM	3	Highest	32
jackson-databind-2.10.2.jar	cpe:2.3:a:fastxml:jackson-databind:2.10.2:*:*:* cpe:2.3:a:fastxml:jackson-modules-java8:2.10.2:*:*:*	pkg:maven/com.fastxml.jackson.core/jackson-databind@2.10.2	HIGH	6	Highest	39
json-path-2.4.0.jar	cpe:2.3:a:json-path:jayway_jsonpath:2.4.0:*:*:*	pkg:maven/com.jayway.jsonpath/json-path@2.4.0	MEDIUM	1	Highest	33
json-smart-2.3.jar	cpe:2.3:a:json-smart_project:json-smart:2.3:*:*:* cpe:2.3:a:json-smart_project:json-smart-v2:2.3:*:*:*	pkg:maven/net.minidev/json-smart@2.3	HIGH	2	Highest	45
log4j-api-2.12.1.jar	cpe:2.3:a:apache:log4j:2.12.1:*:*:*	pkg:maven/org.apache.logging.log4j/log4j-api@2.12.1	LOW	1	Highest	42
logback-core-1.2.3.jar	cpe:2.3:a:qos:logback:1.2.3:*:*:*	pkg:maven/ch.qos.logback/logback-core@1.2.3	HIGH	2	Highest	31
snakeyaml-1.25.jar	cpe:2.3:a:snakeyaml_project:snakeyaml:1.25:*:*:*	pkg:maven/org.yaml/snakeyaml@1.25	CRITICAL	8	Highest	44
spring-boot-2.2.4.RELEASE.jar	cpe:2.3:a:vmware:spring_boot:2.2.4:release:*:*	pkg:maven/org.springframework.boot/spring-boot@2.2.4.RELEASE	CRITICAL	3	Highest	39
spring-core-5.2.3.RELEASE.jar	cpe:2.3:a:pivotal_software:spring_framework:5.2.3:release:*:* cpe:2.3:a:springsource:spring_framework:5.2.3:release:*:* cpe:2.3:a:vmware:spring_framework:5.2.3:release:*:*	pkg:maven/org.springframework/spring-core@5.2.3.RELEASE	CRITICAL*	12	Highest	36
spring-hateoas-1.0.3.RELEASE.jar	cpe:2.3:a:vmware:spring_hateoas:1.0.3:release:*:*	pkg:maven/org.springframework.hateoas/spring-hateoas@1.0.3.RELEASE	MEDIUM	1	Highest	43
spring-web-5.2.3.RELEASE.jar	cpe:2.3:a:pivotal_software:spring_framework:5.2.3:release:*:* cpe:2.3:a:springsource:spring_framework:5.2.3:release:*:* cpe:2.3:a:vmware:spring_framework:5.2.3:release:*:*	pkg:maven/org.springframework/spring-web@5.2.3.RELEASE	CRITICAL*	13	Highest	34
tomcat-embed-core-9.0.30.jar	cpe:2.3:a:apache:tomcat:9.0.30:*:*:* cpe:2.3:a:apache_tomcat:apache_tomcat:9.0.30:*:*:	pkg:maven/org.apache.tomcat.embed/tomcat-embed-core@9.0.30	CRITICAL*	47	Highest	30

6. Functional Testing

Insert a screenshot below of the refactored code executed without errors.



7. Summary

In this project, the Artemis Financial application was refactored to improve secure communication and data integrity in alignment with software security testing protocols. Using the vulnerability assessment process flow, security concerns were identified, analyzed, mitigated, and validated through testing. Key areas addressed during refactoring included secure data transmission, data integrity verification, certificate-based authentication, and dependency vulnerability analysis.

Security improvements were implemented by adding a checksum verification process using the SHA-256 hashing algorithm to ensure that data transferred through the application was not altered or tampered with. Secure communication was established by configuring the application to use HTTPS with a self-signed TLS certificate generated through the Java Keytool. Additionally, static analysis was performed using the OWASP Dependency-Check tool to identify known vulnerabilities in third-party dependencies and confirm that no new vulnerabilities were introduced through the refactoring process.

8. Industry Standard Best Practices

Industry standard best practices for secure coding were applied throughout the refactoring process to mitigate known security vulnerabilities while preserving the application's existing functionality. Modern, NIST-recommended cryptographic algorithms such as AES-256 and SHA-256 were used to replace weaker or outdated security mechanisms. Secure configuration practices were followed by externalizing sensitive settings, such as keystore information, into the application properties file rather than hardcoding them into the source code. The use of HTTPS with TLS aligns with industry standards for protecting data in transit, particularly for financial applications that handle sensitive customer information.

Applying industry standard best practices improves the overall well-being of the organization by reducing the risk of data breaches, regulatory noncompliance, and loss of customer trust. Secure coding practices also enhance long-term maintainability and scalability by ensuring the application can adapt to evolving threats. For Artemis Financial, these practices help protect client data, support business continuity, and reinforce the company's commitment to secure and reliable financial services.

